Distance Learning Management System (DLMS)

Project Overview

A fully functional Web-Based Distance Learning Management System developed using the Waterfall Software Development Life Cycle methodology. The system provides role-based access control for Administrators, Lecturers, and Students with comprehensive course and lecture management capabilities.

Technology Stack

- **Backend:** Python 3.x with Flask Framework
- **Frontend:** HTML5, CSS3, JavaScript
- **Database:** MySQL
- **Libraries:** Flask-MySQLdb, Werkzeug

System Features

Administrator Features

- Create and manage user accounts (Students, Lecturers, Administrators)
- Assign user roles
- Activate/Deactivate user accounts
- View all courses and lectures in the system
- System-wide monitoring and statistics

Lecturer Features

- Secure authentication and dashboard
- Create and manage courses
- Upload video lectures with metadata
- View course statistics
- Manage multiple courses

Student Features

- Secure authentication and dashboard
- Browse available courses
- Enroll in courses
- Access enrolled courses
- Stream video lectures

- Role-based access restrictions

Prerequisites

Before installation, ensure you have:

- Python 3.7 or higher
- MySQL Server 5.7 or higher
- pip (Python package manager)
- A web browser (Chrome, Firefox, Safari, or Edge)

Installation Instructions

Step 1: Install MySQL

1. Download and install MySQL from https://dev.mysql.com/downloads/
2. During installation, set a root password (remember this for configuration)
3. Start the MySQL service

Step 2: Set Up Project Directory

```bash
# Create project directory
mkdir dlms_project
cd dlms_project

# Create folder structure
mkdir templates
mkdir static
mkdir static/css
mkdir static/uploads
mkdir static/uploads/videos
```

Step 3: Install Python Dependencies

```bash
# Install Flask and required packages
pip install flask
pip install flask-mysqldb
pip install werkzeug
```

Step 4: Configure Database

1. Open MySQL command line or MySQL Workbench
2. Run the database.sql file to create the database and tables:

```bash
mysql -u root -p < database.sql
```

Or manually execute the SQL commands from database.sql

Step 5: Configure Application

1. Open `app.py`
2. Update MySQL configuration (lines 11-14):

```python
app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'  # Your MySQL username
app.config['MYSQL_PASSWORD'] = 'your_password'  # Your MySQL password
app.config['MYSQL_DB'] = 'dlms_db'
```

3. Change the secret key (line 8):

```python
app.secret_key = 'your_secure_random_key_here'
```

Step 6: File Structure

Ensure your project has the following structure:

```
dlms_project/
│
├── app.py
├── database.sql
│
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── login.html
│   ├── admin_dashboard.html
│   ├── admin_users.html
│   ├── admin_create_user.html
│   ├── admin_courses.html
│   ├── lecturer_dashboard.html
│   ├── lecturer_courses.html
│   ├── lecturer_create_course.html
│   ├── lecturer_upload_lecture.html
│   ├── lecturer_course_lectures.html
│   ├── student_dashboard.html
│   ├── student_courses.html
│   ├── student_enroll.html
│   ├── student_course_lectures.html
│   └── student_view_lecture.html
│
└── static/
    ├── css/
    │   └── style.css
    └── uploads/
        └── videos/
```

Step 7: Run the Application

```bash
# Navigate to project directory
cd dlms_project

# Run the Flask application
python app.py
```

The application will start on http://127.0.0.1:5000/

Default Login Credentials

Administrator Account

- Email: admin@dlms.com
- Password: admin123

Sample Lecturer Account

- Email: lecturer@dlms.com
- Password: lecturer123

Sample Student Account

- Email: student@dlms.com
- Password: student123

Note: The default passwords are hashed in the database. If you need to create new accounts, use the Administrator interface after logging in.

Usage Guide

For Administrators

1. Log in with admin credentials
2. Navigate to "Manage Users" to create new accounts
3. Create users by providing name, email, password, and role
4. Toggle user activation status as needed
5. View all courses and system statistics

For Lecturers

1. Log in with lecturer credentials
2. Create a course from "Create Course" page
3. Upload video lectures through "Upload Lecture" page
4. Select course, provide lecture details, and upload video file
5. Manage your courses and view lecture statistics

For Students

1. Log in with student credentials
2. Browse available courses in "Enroll in Course"

3. Click "Enroll Now" to join a course
4. Access enrolled courses from "My Courses"
5. Watch video lectures by clicking "Watch Lecture"

Video Upload Specifications

- **Allowed Formats:** MP4, AVI, MOV, WMV, MKV
- **Maximum File Size:** 100MB
- **Storage Location:** static/uploads/videos/
- **Database Storage:** Only file path is stored

Security Features

- Password hashing using Werkzeug's security functions
- Session-based authentication
- Role-based access control with decorators
- SQL injection prevention through parameterized queries
- Secure filename handling for uploads
- CSRF protection through Flask sessions

Testing Checklist

Authentication Tests

- [ ] Login with valid credentials for each role
- [ ] Login with invalid credentials (should fail)
- [ ] Session persistence across pages
- [ ] Logout functionality
- [ ] Access control for restricted pages

Administrator Tests

- [ ] Create new user accounts
- [ ] Toggle user activation status
- [ ] View all courses
- [ ] View system statistics

Lecturer Tests

- [ ] Create new course
- [ ] Upload video lecture
- [ ] View course lectures

- [ ] Verify file upload and storage

Student Tests

- [ ] Enroll in available courses
- [ ] View enrolled courses
- [ ] Access course lectures
- [ ] Stream video content
- [ ] Verify access restrictions (cannot access non-enrolled courses)

Troubleshooting

Database Connection Issues

- Verify MySQL service is running
- Check username and password in app.py
- Ensure database 'dlms_db' exists

File Upload Errors

- Check folder permissions for static/uploads/videos/
- Verify file size is under 100MB
- Confirm file format is supported

Import Errors

- Reinstall Flask dependencies: `pip install -r requirements.txt`
- Verify Python version: `python --version`

Video Playback Issues

- Ensure video file was uploaded successfully
- Check browser compatibility
- Verify video codec is supported

Project Documentation

Waterfall SDLC Phases

Phase 1: Requirements Analysis

- Identified three user roles with distinct functions
- Defined functional and non-functional requirements

- Specified technology stack and database requirements

Phase 2: System Design

- Designed database schema with normalized tables
- Created ER diagram for relationships
- Designed user interface mockups
- Planned folder structure and routing

Phase 3: Implementation

- Developed Flask backend with routes and logic
- Created MySQL database and tables
- Implemented HTML templates with responsive CSS
- Added form validation and file upload handling
- Integrated role-based access control

Phase 4: Testing

- Conducted unit tests on individual routes
- Performed integration testing across modules
- Tested file upload and video streaming
- Verified security and access controls

Phase 5: Deployment

- Prepared deployment instructions
- Created user documentation
- Ready for local demonstration

Future Enhancements

- Email notifications for enrollments
- Quiz and assignment features
- Progress tracking for students
- Discussion forums
- Mobile responsive design improvements
- Cloud deployment capability

Support and Contact

For issues or questions regarding this project, please refer to the documentation or contact the development team.

License

This project is developed for educational purposes as part of an undergraduate project evaluation.

Developed using the Waterfall Software Development Life Cycle © 2025 DLMS Team